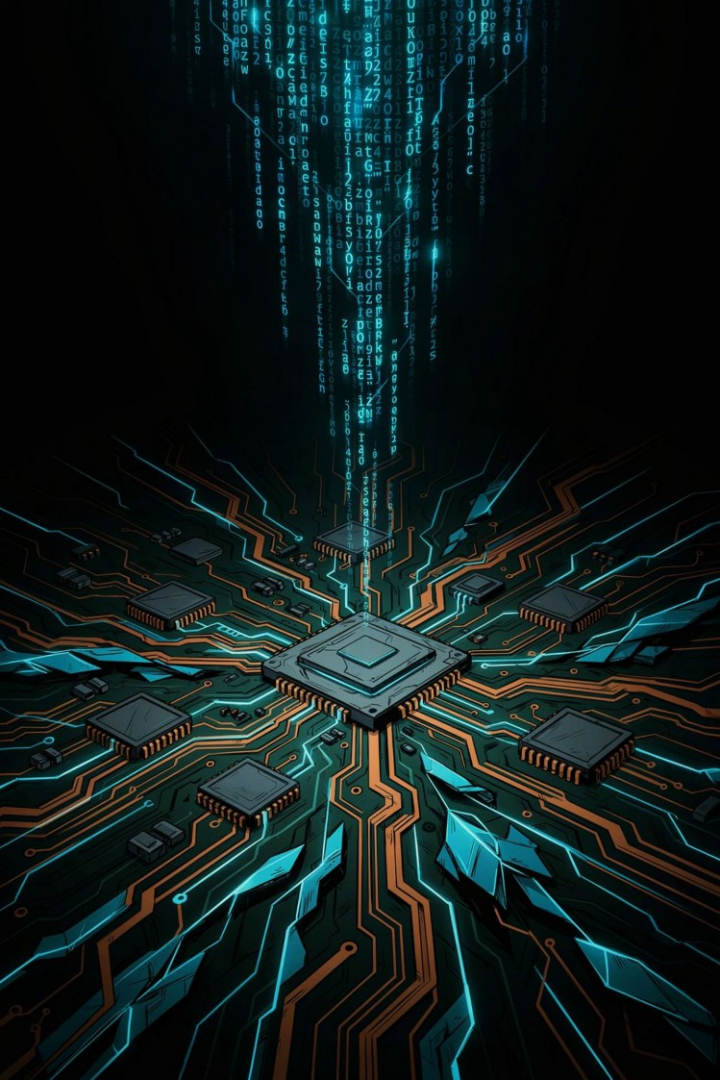




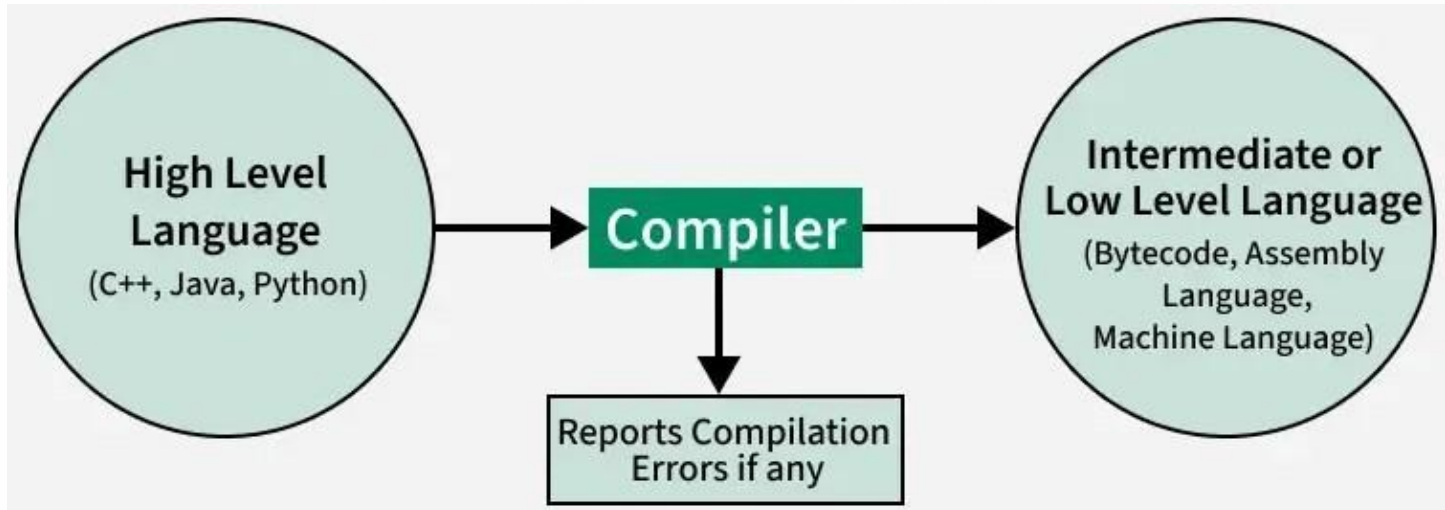
Introdução aos Compiladores

Como um código-fonte se transforma em um programa executável?

Disciplina: Compiladores | **Professores:** Murillo e Alexon

O que é?

Compilador – Interpretador - Montador



Compilador

Converte o código de alto nível em código de máquina

Não executa o código – apenas traduz

Gera um binário (executável) que pode ser rodado posteriormente

Interpretador

Lê e executa o código linha por linha, em tempo de execução

Não gera um executável ou código binário final

Executa o código-fonte diretamente, via ambiente interpretado

Montador (Assembler)

Converte código Assembly (baixo nível) para código de máquina

Não executa o código – apenas monta

Gera código de máquina específico para a arquitetura

COMPILADOR

Código Fonte (ex: C++)

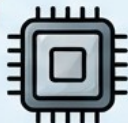
```
int main() {  
    ...  
}
```

Antes da Execução



Arquivo Executável (Binário)

Gravação e Distribuição



Execução Rápida

- Tradução total antes de rodar.
- Gera executável.
- Alta performance.
- Erros encontrados antes da execução.

INTERPRETADOR

Código Fonte (ex: Python)

```
print("Hello")  
print("Hello")  
print("Hello")
```

Durante a Execução



Execução Dinâmica



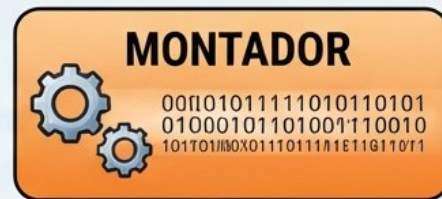
- Tradução e execução linha por linha.
- Não gera executável.
- Desenvolvimento mais rápido.
- Erros encontrados durante a execução.

MONTADOR

Código Assembly

```
MOV AX, 5  
ADD BX, AX
```

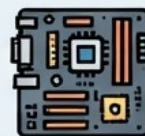
Antes da Execução



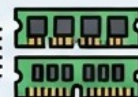
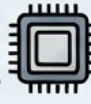
```
00101011111010110101  
01000101101001110010  
10110111010111011111011011
```

Código de Máquina (Objeto)

Tradução Direta de Mnemônicos



Controle Direto de Hardware



- Converte mnemônicos em binário.
- Específico para arquitetura.
- Baixo nível, alta velocidade.

Comparação de Tradutores de Código: Compilador, Interpretador, Montador



Compilador

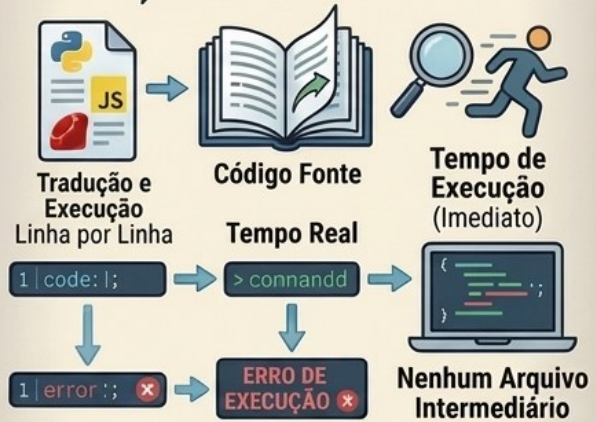


Vantagens:
Execução Rápida,
Código Otimizado

Desvantagens:
Alterações Exigem
Recompilação Completa



Interpretador

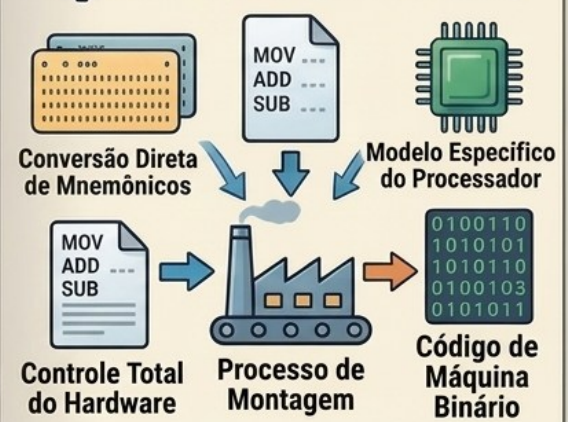


Vantagens:
Desenvolvimento Ágil,
Facilidade para Depuração

Desvantagens:
Execução Mais Lenta
Tradução a Cada Uso



Montador (Assembler)



Vantagens:
Controle Absoluto sobre
Hardware/Memória

Desvantagens:
Código Não Portátil
(Específico de CPU)

Tabela de Comparação Direta

Característica	Compilador	Interpretador	Montador
Entrada Típica	Código de alto nível (C++)	Código de alto nível (Python)	Linguagem Assembly
Saída Gerada	Arquivo executável binário	Nenhum arquivo intermediário	Arquivo binário semântico
Velocidade	Muito rápida ao executar	Mais lenta	Altíssima
Erros	Listados todos ao final	Interrompe na primeira falha	Listados ao final da montagem

Considere a seguinte instrução escrita em uma linguagem de programação:

```
resultado = 10 + 20;
```

**“Quando clicamos em executar,
como o computador entende essa frase?”**

Uma Pergunta Simples

Considere a seguinte instrução escrita em uma linguagem de programação:

```
resultado = 10 + 20;
```

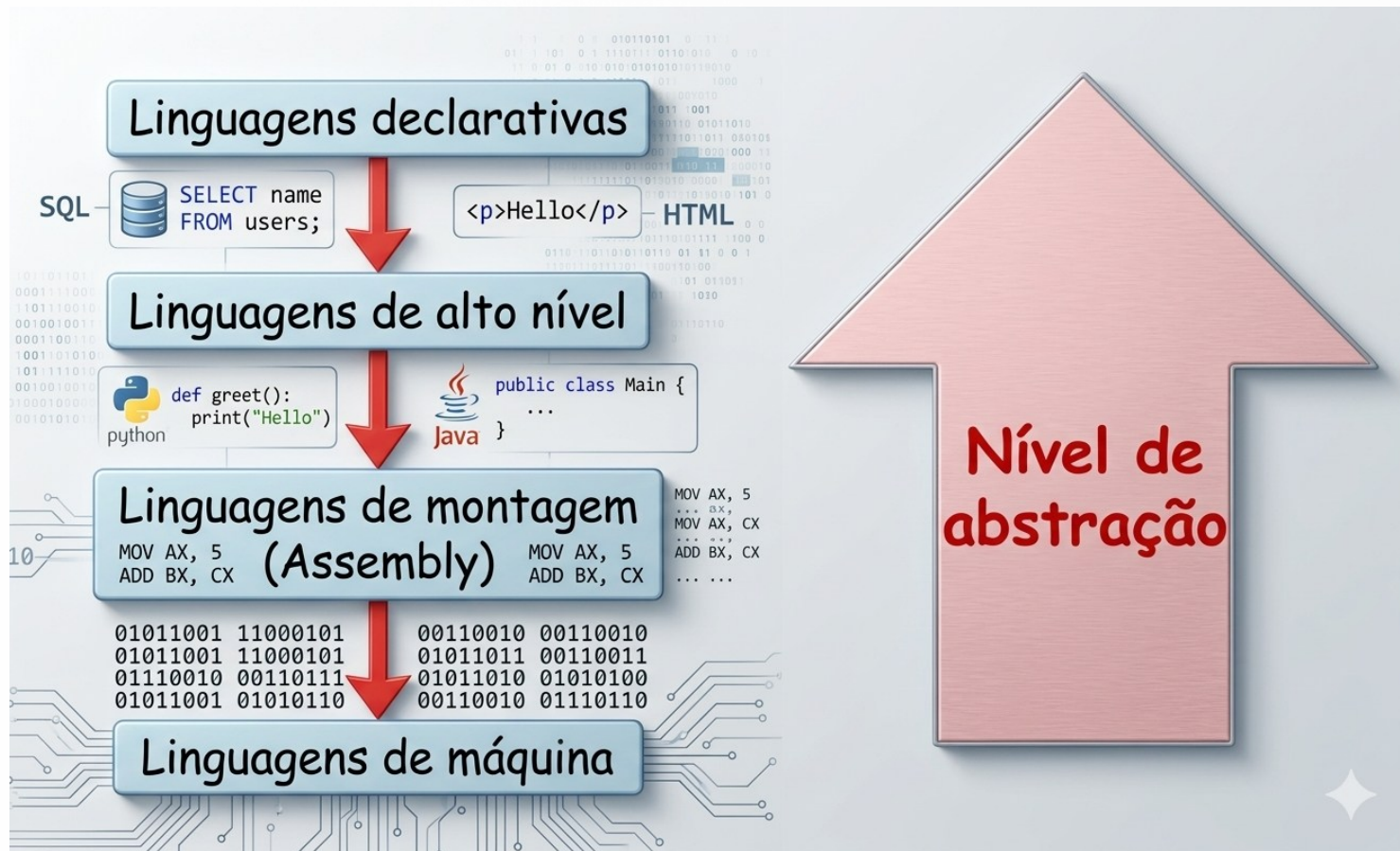
O que o computador vê?

O computador **não entende diretamente** palavras como resultado. Para ele, tudo precisa ser traduzido para **uma linguagem de nível muito mais baixo**.

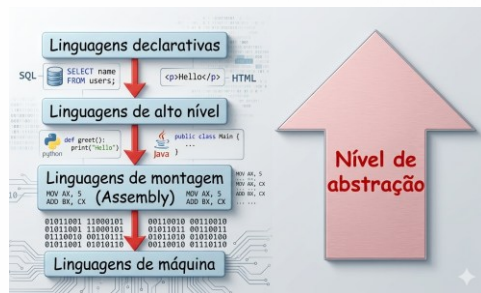
Quem faz essa tradução?

O código precisa ser **analisado**, verificado e traduzido antes de ser executado. Esse trabalho complexo e essencial é realizado por um programa chamado **compilador**.

Níveis de abstração de Linguagens



Níveis de abstração de Linguagens



Opcode (Operation Code / Código de Operação).

Exemplo Prático (Arquitetura x86)

A tradução muda dependendo do processador e dos "alvos" da ação. Veja como o mnemônico MOV se transforma em binário:

Instrução em Assembly:

MOV AL, 61h (Move o valor hexadecimal 61 para o registrador AL)

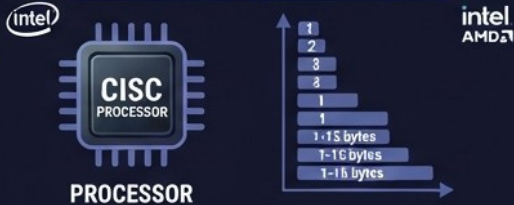
Código de Máquina (Hexadecimal): B0 61

Código de Máquina (Binário): 10110000 01100001

PRINCIPAIS ARQUITETURAS DE CHIPS E ASSEMBLY HOJE

COMPARAÇÃO DE ARQUITETURAS E ASSEMBLY

x86-64 (AMD64)



Arquitetura x86/AMD64
Conjunto de Instruções Complexo (CISC)

Sintaxe Intel:

```
ADD RAX, RBX
```

Sintaxe AT&T:

```
addq %rbx, %rax
```

Soma valores em
RAX e RBX,
resultado em RAX

- Domina PCs e Servidores
- Instruções variáveis
- Registradores de uso geral complexos



ARM (AArch64)

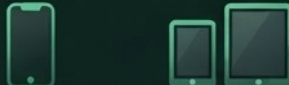


Arquitetura ARMv8/v9
Conjunto de Instruções Reduzido (RISC)
Filosofia Load/Store

```
ADD X0, X1, X2
```

Soma X1 e X2,
resultado em X0

- Domina Smartphones e Tablets
- Alta eficiência energética
- Instruções de tamanho fixo
- Registradores dedicados e limpos



RISC-V (RV64GC)



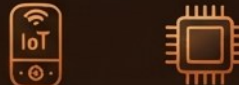
(Open ISA)

Arquitetura Aberta Modular
Baseada em RISC
Extensões (G, C)

```
add x1, x2, x3
```

Soma x2 e x3,
resultado em x1

- Crescente em IoT e IA
- Sem Royalties (Open Source)
- Altamente customizável
- Minimalista





O que é um Compilador?

Um compilador é um programa especializado que atua como intermediário entre o desenvolvedor e a máquina.



Recebe

O código-fonte escrito pelo programador



Verifica

Se o código está corretamente escrito e identifica erros



Traduz

O código para outra linguagem compreendida pela máquina



Viabiliza

A execução do programa pelo processador

Uma Comparação Simples

Podemos comparar o processo de compilação à tradução de uma receita culinária escrita em outro idioma.

Elemento	Comparação com a Compilação
Código-fonte	Receita escrita por uma pessoa, em linguagem humana
Compilador	Tradutor que converte a receita para as instruções da máquina
Código de máquina	Instruções diretamente compreendidas pelo equipamento
Processador	A máquina que executa as instruções traduzidas

Compilador vs. Interpretador

Existem duas abordagens principais para executar um código-fonte. Ambas são amplamente utilizadas na prática.

Compilador

Traduz o programa **antes** de sua execução, gerando um arquivo executável independente.

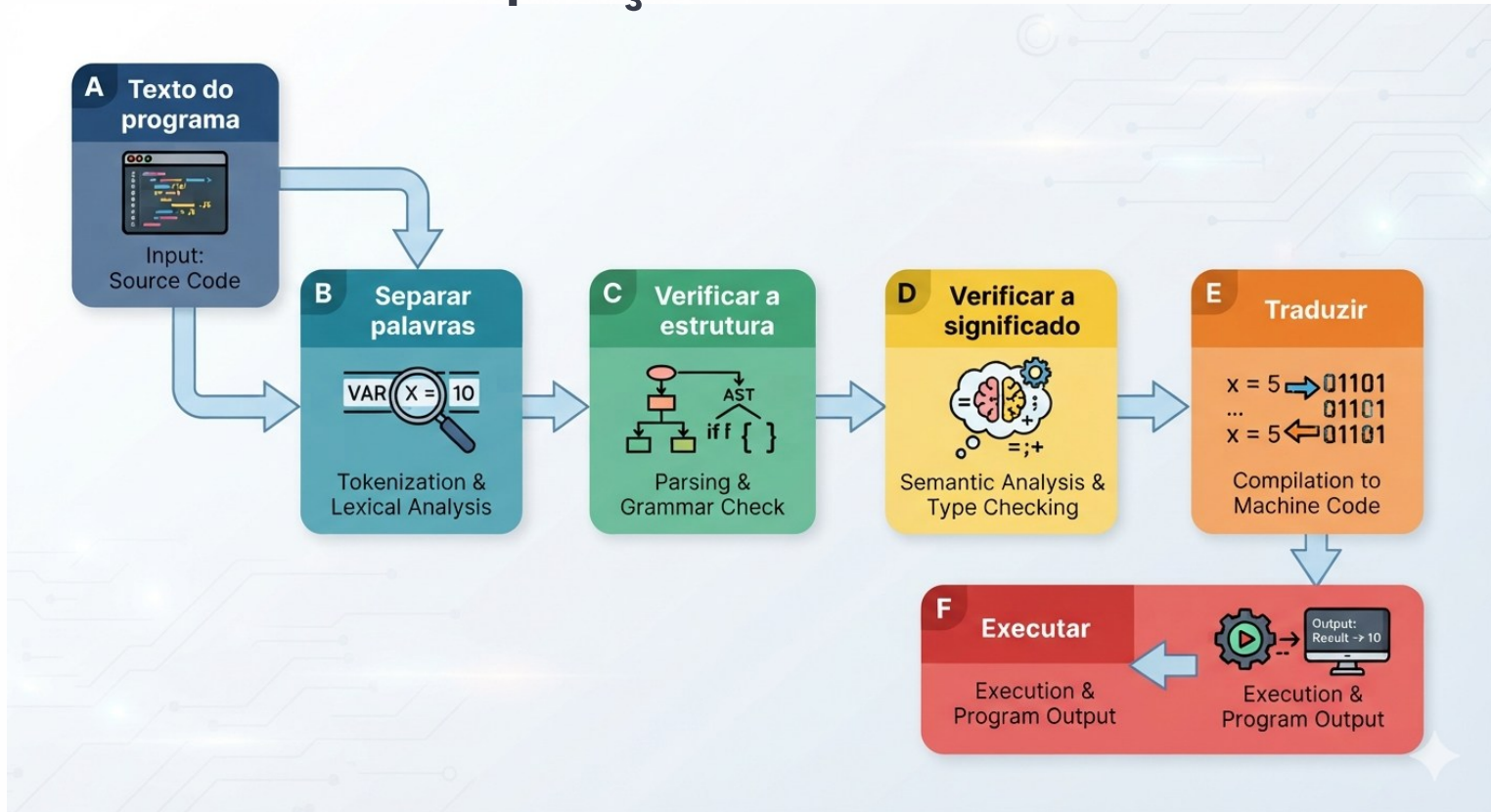
- GCC
- Clang
- Compilador Java (javac)

Interpretador

Analisa e executa o programa **linha por linha**, durante seu próprio processamento, sem gerar um executável separado.

- Python
- Ruby
- JavaScript (em browsers)
- JVM
- PVM

O Processo de Compilação



O Processo de Compilação

A compilação não é uma operação única — ela é dividida em etapas encadeadas, cada uma com uma responsabilidade bem definida.



Cada fase recebe a saída da fase anterior e produz uma representação mais refinada do programa, até chegar ao código executável.

O que um Compilador Gera?

(Compilação para Diferentes Plataformas)



O que um Compilador Gera?

Tabela Detalhada de Resultados

```
main.c x
int main() {
    printf("Olá, Mundo\n");
    return 0;
}
```

Código Fonte
(Source Code)
Editor de Código

Compilador
(Compiler)

**Processo de
Compilação**

Sistema/ Ambiente	Extensão Comum do Resultado	Tipo de Arquivo
 Windows (x64)	 programa.exe	Binário Nativo (PE)
 Linux	 ou  programa / programa.bin	Binário Nativo (ELF)
 macOS	 ou  programa / programa.app	Binário Nativo (Mach-O)
 Máquina Virtual (Java JVM)	 Programa.class / .jar	Bytecode Intermediário
 Web (Navegadores)	 programa.wasm	WebAssembly

O EXECUTÁVEL FINAL: SISTEMA OPERACIONAL VS. COMPILADOR

Diferenciando Regras, Formato e Execução

1. REGRAS DO COMPILADOR



2. O PLANO DO ARQUITETO



3. REGRAS DO SISTEMA OPERACIONAL (O DECISOR)



4. O EXECUTÁVEL FINAL

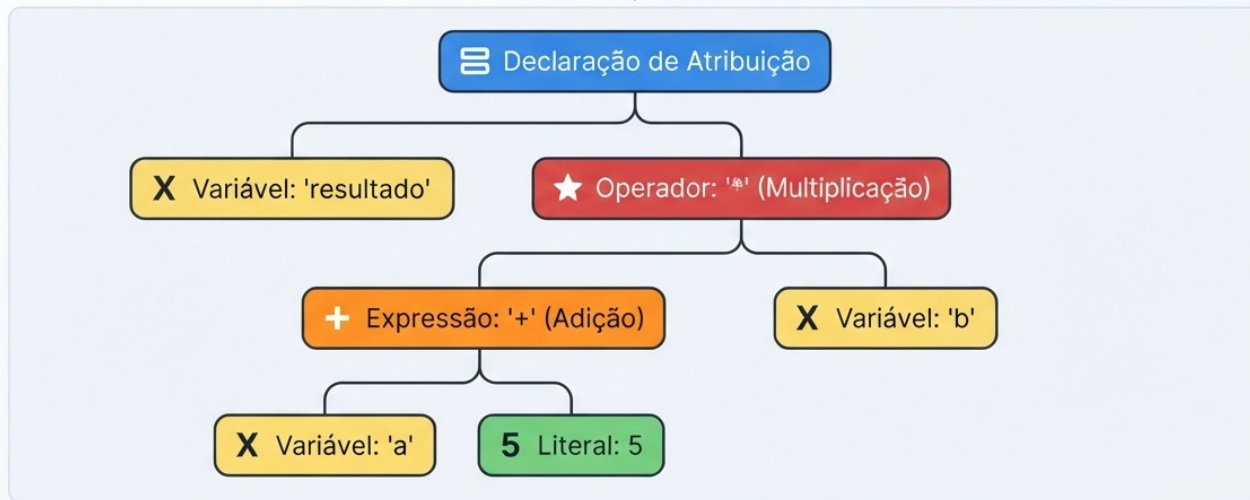


Exemplo de AST

C Source

```
10 int resultado = (a + 5) * b;
```

Análise Sintática e Semântica (Parser)



Árvore Sintática Abstrata (AST) da expressão: `int resultado = (a + 5) * b;`

Exemplo de AST

A **AST (Árvore Sintática Abstrata)** é utilizada porque o texto puro do seu código é unidimensional (uma linha atrás da outra), enquanto o pensamento computacional e as regras gramaticais são **hierárquicos e multidimensionais**

A AST remove os detalhes inúteis do texto (como parênteses, chaves e ponto-e-vírgula) e foca apenas na **relação estrutural e na ordem de precedência** das operações.

Como a AST é Criada no Pipeline

A criação da AST ocorre na metade inicial do ciclo de vida de um compilador (conhecido como *Front-End*):

1. **Análise Léxica (Scanner):** Lê o texto bruto caractere por caractere e o agrupa em palavras-chave significativas (*tokens*).
2. **Análise Sintática (Parser):** Organiza os tokens em regras gramaticais e monta a **AST**.
3. **Análise Semântica:** Percorre a AST para verificar erros de lógica, como somar um texto com um número (erros de tipo).



As Partes de um Compilador

Frontend

- Análise léxica
- Análise sintática
- Análise semântica

Middle-end

- Representação intermediária (IR)
- Otimização do código

Backend

- Geração de assembly
- Geração de bytecode
- Geração de código de máquina

A Linguagem Experimental MiniLang

Durante as aulas, construiremos juntos um compilador completo para uma linguagem didática chamada **MiniLang**. Ela foi projetada para ser simples o suficiente para estudarmos, mas real o suficiente para ser funcional.

Código MiniLang

```
variavel idade = 30;  
variavel dobro = idade * 2;  
escreva(dobro);
```

Resultado Esperado

60

A MiniLang usa palavras em português para facilitar o entendimento das estruturas da linguagem durante o aprendizado.

O que a MiniLang Deverá Fazer?

A primeira versão da linguagem já será capaz de realizar operações reais e úteis. Funcionalidades adicionais serão incorporadas ao longo do semestre.

1 Declarar variáveis

Utilizando a palavra reservada `variavel`

2 Armazenar números inteiros

Com atribuição direta de valores

3 Realizar operações matemáticas

Adição, subtração, multiplicação e divisão

4 Mostrar resultados

Com o comando `escreva()`

FASE 1

Análise Léxica

A análise léxica é a primeira etapa do compilador. Ela percorre o código-fonte e o divide em pequenas unidades chamadas **tokens** — os blocos fundamentais da linguagem.

Código de entrada

```
variavel idade = 30;
```

Separação em tokens

```
variavel | idade | = | 30 | ;
```

Cada unidade tem um tipo e um papel específico na gramática da linguagem.

O que é um Token?

Um token é a classificação de um elemento do código. O **lexer** (analisador léxico) é o componente responsável por produzir esses tokens a partir do texto bruto do programa.

Texto no Código	Token Gerado
variavel	VARIAVEL
idade	IDENTIFICADOR
=	ATRIBUICAO
30	NUMERO
;	PONTO_E_VIRGULA

Como o Lexer Trabalha?

O lexer percorre o código-fonte **caractere por caractere**, tomando decisões baseadas no que encontra. Esse processo é chamado de varredura linear.

Encontrou um espaço

Ignorar e avançar para o próximo caractere

Encontrou uma letra

Continuar lendo até formar uma palavra completa (palavra reservada ou identificador)

Encontrou um número

Continuar lendo até formar o valor numérico completo

Encontrou um operador

Criar imediatamente o token correspondente (=, +, ;)

Caractere desconhecido

Informar um erro léxico com a posição no código

Exemplo Completo de Tokenização

Veja como o lexer processa duas linhas de código MiniLang e produz a lista de tokens correspondente:

Entrada

```
variavel nota = 8;  
escreva(nota);
```

Saída – Lista de Tokens

```
VARIAVEL  
IDENTIFICADOR(nota)  
ATRIBUICAO  
NUMERO(8)  
PONTO_E_VIRGULA  
ESCREVA  
ABRE_PARENTESES  
IDENTIFICADOR(nota)  
FECHA_PARENTESES  
PONTO_E_VIRGULA
```

O que é um Erro Léxico?

Um erro léxico ocorre quando o lexer encontra um caractere que **não faz parte do alfabeto** da linguagem — ou seja, um símbolo que a MiniLang simplesmente não reconhece.

Código com Erro

```
variavel idade = 30 @ 2;
```

Mensagem de Erro

Erro léxico na linha 1:
caractere '@' não reconhecido.

O símbolo @ não pertence ao conjunto de caracteres válidos da MiniLang. O compilador deve interromper a análise e informar o problema com clareza.

Atividade: Encontre os Tokens

Analise o programa abaixo e identifique cada token, classificando-o em uma das cinco categorias:

```
variavel total = 10 + 5;  
escreva(total);
```

1. Palavras reservadas

Ex.: `variavel`, `escreva`

2. Identificadores

Nomes criados pelo
programador

3. Números

Literais numéricos inteiros

4. Operadores

Símbolos de operação e
atribuição

5. Pontuação

Parênteses e ponto e vírgula

FASE 2

Análise Sintática

Enquanto o lexer verifica **quais** palavras existem no código, o **parser** (analisador sintático) verifica se essas palavras aparecem **na ordem correta**.

✓ Correto

```
variavel idade = 30;
```

Estrutura na sequência esperada pela gramática

✗ Incorreto

```
idade variavel 30 =;
```

Os tokens existem, mas estão em uma **ordem inválida** — erro sintático!

Uma Comparação com a Linguagem Humana

O mesmo princípio que o parser aplica ao código também vale para a língua portuguesa. Palavras corretas, em ordem errada, perdem o sentido.

✓ Frase compreensível

"O aluno realizou a atividade."

Palavras corretas, ordem correta — sentido claro.

✗ Palavras em ordem incorreta

"Atividade o realizou aluno a."

As palavras existem, mas a organização está errada — impossível de compreender.

📄 O parser não se preocupa com o significado do código — apenas com a estrutura. O significado é responsabilidade da análise semântica.

Regra de uma Declaração na MiniLang

Uma declaração válida da MiniLang deve seguir uma estrutura rígida definida pela gramática da linguagem:

VARIAVEL IDENTIFICADOR = EXPRESSAO ;

✓ Exemplo Válido

```
variavel idade = 30;
```

✗ Exemplos Inválidos

```
variavel = 30;  
variavel idade 30;  
variavel idade = ;
```

Em cada caso, a gramática é violada: falta o identificador, falta o `=` ou falta a expressão.

O que é um Erro Sintático?

Um erro sintático ocorre quando os tokens estão em uma ordem inválida ou quando algum elemento obrigatório está ausente na estrutura do programa.

Código com Erro

```
variavel idade = 30
```

O ponto e vírgula final está faltando.

Mensagem de Erro

Erro sintático na linha 1:
esperado ';' após
a declaração.

Boas mensagens de erro indicam **o que** estava esperado e **onde** o problema ocorreu.

Atividade: Válido ou Inválido?

Classifique cada instrução abaixo como **válida** ou **inválida**. Para as inválidas, identifique o elemento ausente, o elemento fora de ordem e proponha uma mensagem de erro clara.

Instrução	Classificação	Problema
<code>variavel x = 10;</code>		
<code>variavel = 10;</code>		
<code>escreva(x);</code>		
<code>escreva x;</code>		
<code>variavel y 20;</code>		

Atividade: Válido ou Inválido?

Classifique cada instrução abaixo como **válida** ou **inválida**. Para as inválidas, identifique o elemento ausente, o elemento fora de ordem e proponha uma mensagem de erro clara.

Instrução	Classificação	Problema
<code>variavel x = 10;</code>	✓ Válida	—
<code>variavel = 10;</code>	✗ Inválida	Falta identificador
<code>escreva(x);</code>	✓ Válida	—
<code>escreva x;</code>	✗ Inválida	Faltam parênteses
<code>variavel y 20;</code>	✗ Inválida	Falta o operador =

FASE 3

Construção da AST

AST significa **Árvore Sintática Abstrata** (Abstract Syntax Tree). Ela representa a estrutura lógica e hierárquica de um programa, capturando as relações entre os elementos — especialmente a **precedência de operadores**.

Expressão

`10 + 5 * 2`

A multiplicação deve ser realizada **antes** da soma, conforme as regras matemáticas.

Por que a árvore importa?

A AST codifica a ordem correta das operações de forma explícita, eliminando ambiguidades antes da geração de código.

AST de uma Expressão

Veja como a AST representa a precedência entre operadores na expressão `10 + 5 * 2`. A multiplicação está mais funda na árvore, indicando que deve ser calculada primeiro.

Estrutura da Árvore

Soma

├─ Número: 10

└─ Multiplicação

├─ Número: 5

└─ Número: 2

Cálculo Resultante

`10 + (5 × 2)`

`= 10 + 10`

`= 20`

A árvore registra e preserva a precedência das operações de forma estruturada e inequívoca.

Os Parênteses Mudam a Árvore

Quando usamos parênteses, a estrutura da AST muda — e o resultado final também. Isso demonstra como a árvore captura fielmente a intenção do programador.

Expressão com parênteses

```
(10 + 5) * 2
```

Nova Árvore

Multiplicação

```
├─ Soma
│  ├── Número: 10
│  └── Número: 5
└─ Número: 2
```

Cálculo Resultante

```
(10 + 5) × 2
= 15 × 2
= 30
```

Sem parênteses o resultado era **20**. Com parênteses, **30**. A AST captura exatamente essa diferença de intenção.

Atividade: Construa a AST

Para cada expressão abaixo, desenhe a árvore sintática abstrata correspondente e depois calcule o resultado. Use as regras de precedência matemática (* e / antes de + e -).

Expressão 1

$8 + 2 * 3$

Resultado: ?

Expressão 2

$4 * 5 + 2$

Resultado: ?

Expressão 3

$10 - 6 / 2$

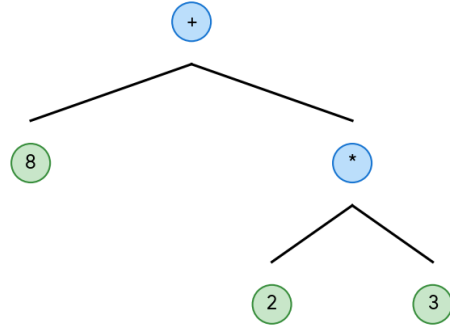
Resultado: ?

Expressão 4

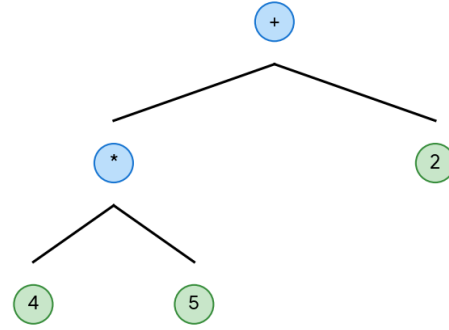
$(10 - 6) / 2$

Resultado: ?

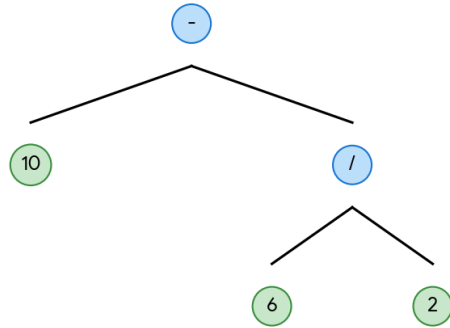
1) $8 + 2 * 3$



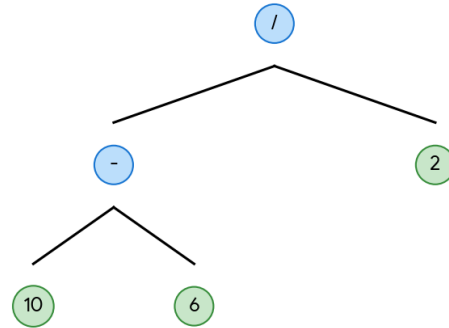
2) $4 * 5 + 2$



3) $10 - 6 / 2$



4) $(10 - 6) / 2$



Análise Semântica

A análise semântica verifica se o programa **faz sentido** — não apenas se está bem formado. Uma estrutura sintaticamente correta ainda pode conter erros de significado.

Código sintaticamente correto

```
escreva(idade);
```

A estrutura está perfeita: palavra reservada, parênteses, identificador, ponto e vírgula.

Mas e o significado?

Se `idade` nunca foi declarada anteriormente, o programa contém um **erro semântico**. A análise semântica detecta exatamente esse tipo de problema.

Sintaxe Correta Não Garante Significado

Assim como em linguagem natural, é possível construir uma frase gramaticalmente perfeita que não faz sentido no mundo real.



Na Linguagem Humana

"A cadeira resolveu a equação."

Estrutura gramatical **válida**, mas semanticamente **absurda**.



Na Programação

```
variavel idade = 30;  
escreva(nome);
```

nome nunca foi declarado. Sintaxe correta, semântica **inválida**.

Exemplos de Erros Semânticos

A análise semântica detecta três categorias principais de problemas que escapam à análise sintática:

Variável não declarada

```
escreva(idade);
```

Uso de uma variável que nunca foi definida no programa

Declaração duplicada

```
variavel idade = 30;  
variavel idade = 40;
```

Tentativa de declarar a mesma variável duas vezes no mesmo escopo

Tipos incompatíveis

```
variavel idade = 30;  
idade = "Murillo";
```

Atribuição de um texto a uma variável declarada como inteiro

Tabela de Símbolos

A tabela de símbolos funciona como uma **planilha interna** do compilador. Ela registra informações sobre todas as variáveis e estruturas declaradas no programa, sendo consultada durante toda a análise semântica.

Nome	Tipo	Valor Inicial	Linha
idade	inteiro	30	1
nota	inteiro	8	2

Como a Tabela de Símbolos é Utilizada?

A tabela é consultada e atualizada constantemente durante a análise do programa. Cada operação tem uma consequência diferente sobre ela.

→ **Ao encontrar** `variavel idade`
`= 30;`

O compilador **registra** a entrada `idade` na tabela com tipo inteiro e valor 30

→ **Ao encontrar** `escreva(idade);`

O compilador **consulta** a tabela e confirma que `idade` existe — tudo certo

→ **Ao encontrar** `escreva(nome);`

O compilador verifica que `nome` **não existe** na tabela — erro semântico!

Geração de Código

Após analisar léxica, sintática e semanticamente o programa, o compilador precisa **traduzir** o código MiniLang para outra linguagem que possa ser executada.

Código MiniLang

```
variavel idade = 30;  
escreva(idade);
```

Código C Gerado

```
int idade = 30;  
printf("%d\n", idade);
```

A MiniLang é traduzida para C válido, que depois pode ser compilado pelo GCC.

Por que Gerar Código C?

Gerar código de máquina diretamente é extremamente complexo — envolve registradores, endereçamento de memória e especificidades de cada arquitetura. A abordagem em duas etapas nos permite focar no que importa.

01

MiniLang → C

Nosso compilador traduz o código MiniLang para código C legível e válido

02

C → Executável

O GCC compila o arquivo C gerado e produz o programa executável final

03

Aprendizado sem Assembly

Estudamos compiladores com foco nas fases de análise, sem nos perder nos detalhes do hardware

REGISTRADORES E LLVM

Gerar código de máquina diretamente é extremamente complexo — envolve registradores, endereçamento de memória e especificidades de cada arquitetura. A abordagem em duas etapas nos permite focar no que importa.

As Duas Etapas do Compilador

1. Front-end (Independente de Máquina)

- Analisa o código que você escreveu.
- Verifica erros de sintaxe e regras da linguagem.
- Traduz o código para uma **Representação Intermediária (IR)**.
- A IR funciona como um código "universal" e simplificado.

2. Back-end (Dependente de Máquina)

- Pega essa Representação Intermediária (IR).
- Otimiza o código para rodar mais rápido.
- Traduz a IR para o código de máquina específico do processador (gerenciando registradores e memória).

REGISTRADORES E LLVM

Gerar código de máquina diretamente é extremamente complexo — envolve registradores, endereçamento de memória e especificidades de cada arquitetura. A abordagem em duas etapas nos permite focar no que importa.

Exemplo Prático: O Ecossistema LLVM / Clang

O exemplo moderno mais famoso dessa abordagem é o **LLVM**, utilizado por linguagens como Swift, Rust, C e C++.

Multiplicidade: Se você tem 3 linguagens (C, Rust, Swift) e 3 arquiteturas de chip (x86, ARM, RISC-V).

Sem a IR: Você precisaria criar 9 compiladores diferentes (um para cada combinação de linguagem e chip).

Com a IR: Cada linguagem possui apenas um Front-end que gera a IR do LLVM. O LLVM possui um Back-end para cada chip que lê essa mesma IR. Você só precisa programar 3 Front-ends e 3 Back-ends (6 componentes no total).

LLVM

O **LLVM** é um conjunto de ferramentas e bibliotecas modular para criar compiladores. Ele traduz códigos de linguagens de programação em instruções otimizadas para diferentes tipos de processadores. O nome original era *Low Level Virtual Machine*, mas hoje ele funciona como uma infraestrutura genérica de compilação.

Como o LLVM funciona

- **Front-end:** Lê o código-fonte da sua linguagem e o converte para uma linguagem universal chamada *Representação Intermediária* (IR).
- **Otimizador:** Melhora o código intermediário para que ele rode mais rápido e use menos memória.
- **Back-end:** Transforma o código intermediário otimizado em código de máquina real para chips específicos (como x86, ARM ou RISC-V).

Por que ele é importante

- **Reutilização:** Criadores de novas linguagens não precisam criar um compilador do zero para cada processador.
- **Desempenho:** Oferece ferramentas avançadas de otimização de código prontas para uso.
- **Popularidade:** É a base de projetos famosos como o compilador **Clang** (para C e C++) e alimenta linguagens como **Rust**, **Swift** e **Julia**.

O QUE É UMA IR?

Uma **IR (Intermediate Representation ou Representação Intermediária)** é um código gerado pelo compilador que serve como um "meio-termo" entre a linguagem de alto nível que você escreve (como C, Rust ou Python) e o código de máquina binário executado pela CPU.

A IR funciona como uma linguagem de montagem (Assembly) idealizada, universal e simplificada, que não depende de nenhum processador real.

Como ela se parece na prática?

O formato mais famoso do mundo é a **LLVM IR**. Ela utiliza uma estrutura chamada **SSA (Static Single Assignment)**, onde cada variável só pode ser definida uma única vez, facilitando a otimização pelo compilador.

O QUE É UMA IR?

A Representação Intermediária (LLVM IR)

O compilador transforma o código acima nesta estrutura textual e lógica:

llvm

Seu código original (C ou C++)

```
int calcular(int a, int b) {  
    int x = a + b;  
    return x * 2;  
}
```

```
define i32 @calcular(i32 %a, i32 %b) {  
    entry:  
        ; %1 armazena o resultado de a + b (i32 significa inteiro de 32  
        bits)  
        %1 = add nsw i32 %a, %b  
  
        ; %2 armazena o resultado da multiplicação por 2  
        %2 = mul nsw i32 %1, 2  
  
        ; Retorna o valor final  
        ret i32 %2  
}
```


O QUE É UMA IR?

O Código de Máquina Final (Assembly x86 para Intel/AMD)

O Back-end lê a IR acima e a traduz para o hardware real, gerenciando os registradores (edi, esi, eax):

Assembly

calcular:

mov eax, edi ; Move o argumento 'a' para o registrador eax

add eax, esi ; Soma o argumento 'b' em eax ($eax = a + b$)

add eax, eax ; Multiplica por 2 somando o número a ele mesmo (otimização de hardware)

ret ; Retorna o valor contido em eax

O QUE É UMA IR?

Por que a IR é fundamental?

Otimização Universal: É muito mais fácil fazer o computador identificar códigos redundantes ou loops desnecessários olhando para a IR limpa do que para o código C original ou para o Assembly complexo.

Portabilidade: Se você inventar uma linguagem nova hoje, basta criar um programa que transforme sua linguagem em **LLVM IR**. Automaticamente, seu código poderá rodar em Intel, AMD, ARM, Apple Silicon e qualquer outro chip compatível com o ecossistema LLVM.

ONDE FICA OS REGISTRADORES?

Os **registradores** são as memórias mais rápidas que existem em um computador. Eles ficam localizados **dentro do próprio núcleo (core) do processador (CPU)**.

Diferente da memória RAM, que fica em placas separadas na placa-mãe, os registradores fazem parte do circuito elétrico imediato onde as operações matemáticas e lógicas acontecem.

A CPU não consegue fazer contas diretamente com dados que estão na memória RAM. Para somar dois números, ela precisa primeiro "copiar" esses números para os seus bolsos internos. Esses bolsos são os registradores.

Pense neles com a seguinte analogia de um escritório:

- **Memória RAM:** É o arquivo de metal no fundo da sala. Guarda muita coisa, mas você demora para ir até lá buscar.
- **Memória Cache:** É a gaveta da sua mesa. Guarda o que você usa com frequência.
- **Registradores:** É a sua **própria mão**. Você só consegue mexer ou transformar o que estiver segurando agora.

Endereços (dec | binário) MP

255		11111111		101111110
254		11111110		01000101
253		11111101		01100101
252		11111100		10110101
251		11111011		00000101
250		11111010		01110110
249		11111001		00001101
248		11111000		00010000
247		00010111		00010111
246		11110110		X X X X X X X X

12		00001100		X X X X X X X X
11		00001011		X X X X X X X X
10		00001010		X X X X X X X X
9		00001001		X X X X X X X X
8		00001000		X X X X X X X X
7		00001001		X X X X X X X X
6		00001010		X X X X X X X X
5		00000101		X X X X X X X X
4		00000010		X X X X X X X X
3		00000001		X X X X X X X X
2		00000010		X X X X X X X X
1		00000001		X X X X X X X X
0		00000000		X X X X X X X X

Controlador da MP 

Barramento de:

 dados

 Endereços

 Controle

 Barramento de Alimentação

Ponteiro de instruções 

Registrador de instruções 

REM: Guarda o endereço

RDM

Barramento de Dados

Barramento de Endereços

Barramento de Controle

Barramento de Alimentação

Unidade de Controle

Registradores

ULA

ULA: Faz cálculos

Microprocessador (UCP)

Cérebro do Computador

ONDE FICA OS REGISTRADORES?

Exemplo Prático de Nomes e Uso

No exemplo da resposta anterior, usamos três registradores reais de um processador Intel/AMD: %edi, %esi e %eax. Veja o que aconteceu no circuito da CPU:

O sistema colocou o primeiro número no registrador **EDI**.

O sistema colocou o segundo número no registrador **ESI**.

A CPU copiou o valor de EDI para o registrador **EAX** (mov eax, edi).

A CPU mandou a ULA somar o valor de ESI dentro de EAX (add eax, esi).

O resultado final ficou guardado em **EAX**, que por padrão é o registrador usado em computadores para entregar a resposta de uma função.



OBSERVAÇÕES:

**TODA SEXTA-FEIRA TEREMOS ATIVIDADES INDIVIDUAIS;
VALERÃO PONTOS PARA SOMAR À NOTA;
MATERIAL DE APOIO: NOTEBOOK INDIVIDUAL.**

P2 = 50% Projeto Final e 50% somatório de atividades individuais.

PROJETO FINAL = duplas selecionadas pelo professor.

**Instruções para o
Compilador 1
Faça o fork repos
do Git do professor.**





Compilador 1

Analizando um Compilador Real – (5/50% - pontuação)

Antes de desenvolver nosso próprio projeto, você analisará um pequeno compilador disponível no GitHub. Esse estudo ajudará a compreender como as peças se encaixam na prática.

Projeto sugerido: [mini-c-compiler](#)

Lexer

Análise léxica
completa

Parser

Análise sintática

AST

Representação em
árvore

Erros

Tratamento de erros

Como Começar a Análise do Repositório?

Não é necessário compreender cada linha de código. O objetivo é identificar a estrutura geral e localizar os componentes principais do compilador.

01

Leia o README

Entenda o propósito do projeto e como configurá-lo

02

Identifique o arquivo principal

Localize o ponto de entrada do compilador

03

Procure o lexer e o parser

Identifique onde cada fase está implementada

04

Localize a AST e os testes

Veja como a árvore é representada e quais testes existem

05

Observe o tratamento de erros

Entenda como o projeto reporta problemas ao usuário

Executando o Projeto

Siga os passos abaixo para clonar e executar o mini-c-compiler em sua máquina. Consulte o README do repositório para instruções específicas de compilação.

1. Clonar o repositório

```
git clone https://github.com/  
ironrinox/mini-c-compiler.git
```

2. Entrar na pasta

```
cd mini-c-compiler
```

Próximos passos

Após clonar, siga as instruções de compilação apresentadas no README do projeto. Em geral, será necessário um compilador C instalado (GCC ou Clang) e, possivelmente, o make.



Referências Práticas

Estes recursos complementam as aulas e permitem aprofundar os conceitos de forma prática, com código real e exemplos funcionais.

Crafting Interpreters

Livro gratuito e completo sobre compiladores e interpretadores, com implementação passo a passo. craftinginterpreters.com | [Repositório no GitHub](#)

Mini C Compiler

Projeto analisado em aula — pequeno compilador C com lexer, parser e AST.
github.com/ironrinox/mini-c-compiler

chibicc – compilador C minimalista

Um compilador C pequeno mas surpreendentemente completo, ótimo para estudar geração de código real. github.com/rui314/chibicc

Tutorial Kaleidoscope – LLVM

Tutorial oficial do LLVM que ensina a construir um compilador completo para uma linguagem funcional. llvm.org/docs/tutorial

